try1

```
# ============================================================================
# ANÁLISIS PROFESIONAL DE TEXTO - HISTORIAS DE FANTASMAS
# Clasificación de historias paranormales usando Naive Bayes (optimizado)
# ============================================================================

# --- Librerías ---
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✓ dplyr     1.1.4     ✓ readr     2.1.5
✓ forcats   1.0.0     ✓ stringr   1.5.2
✓ ggplot2   3.5.2     ✓ tibble    3.3.0
✓ lubridate 1.9.4     ✓ tidyr     1.3.1
✓ purrr     1.1.0
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidytext)
library(Matrix)
```

```
Adjuntando el paquete: 'Matrix'

The following objects are masked from 'package:tidyr':

    expand, pack, unpack
```

```
library(caret)
```

```
Cargando paquete requerido: lattice

Adjuntando el paquete: 'caret'

The following object is masked from 'package:purrr':

    lift
```

```
library(naivebayes)
```

```
naivebayes 1.0.0 loaded
For more information please visit:
https://majkamichal.github.io/naivebayes/
```

```
library(e1071)    # solo para confusionMatrix si no está cargado por caret
library(janitor)
```

```
Adjuntando el paquete: 'janitor'

The following objects are masked from 'package:stats':

    chisq.test, fisher.test
```

```
library(ggplot2)
library(viridis)
```

```
Cargando paquete requerido: viridisLite
```

```
library(scales)
```

```

Adjuntando el paquete: 'scales'

The following object is masked from 'package:viridis':

    viridis_pal

The following object is masked from 'package:purrr':

    discard

The following object is masked from 'package:readr':

    col_factor
```

```
options(width = 110)

cat("=== CARGANDO DATASET DE HISTORIAS DE FANTASMAS ===\n")
```

```
=== CARGANDO DATASET DE HISTORIAS DE FANTASMAS ===
```

```
# --- 1) Carga del dataset ---
stories_df <- read.csv("ghost_stories_2000.csv",
                       stringsAsFactors = FALSE,
                       encoding = "UTF-8")
```

```
# --- 2) Exploración básica ---
cat("Dimensiones del dataset:", nrow(stories_df), "filas,", ncol(stories_df), "columnas\n")
```

```
Dimensiones del dataset: 2201 filas, 4 columnas
```

```
cat("Columnas:", paste(names(stories_df), collapse = ", "), "\n\n")
```

```
Columnas: title, story, category, location
```

```
print("=== RESUMEN DEL DATASET ===")
```

```
[1] "=== RESUMEN DEL DATASET ==="
```

```
print(summary(stories_df))
```

```
    title              story             category           location
 Length:2201        Length:2201        Length:2201        Length:2201
 Class :character   Class :character   Class :character   Class :character
 Mode  :character   Mode  :character   Mode  :character   Mode  :character
```

```
# Métricas de tamaño de texto
stories_df <- stories_df %>%
  mutate(
    word_count     = str_count(story, "\\S+"),
    char_count     = nchar(story),
    sentence_count = str_count(story, "[.!?]+")
  )

print("=== ESTADÍSTICAS DE CONTENIDO ===")
```

```
[1] "=== ESTADÍSTICAS DE CONTENIDO ==="
```

```
print(stories_df %>%
        select(word_count, char_count, sentence_count) %>%
        summary())
```

```
  word_count       char_count    sentence_count
 Min.   :  88.0   Min.   :  454   Min.   :  3.00
 1st Qu.: 283.0   1st Qu.: 1424   1st Qu.: 17.00
 Median : 412.0   Median : 2085   Median : 25.00
 Mean   : 539.3   Mean   : 2763   Mean   : 32.97
 3rd Qu.: 639.0   3rd Qu.: 3276   3rd Qu.: 39.00
 Max.   :4973.0   Max.   :26466   Max.   :261.00
```

```
# --- 3) Distribuciones descriptivas ---
cat("\n=== DISTRIBUCIÓN DE CATEGORÍAS ===\n")
```

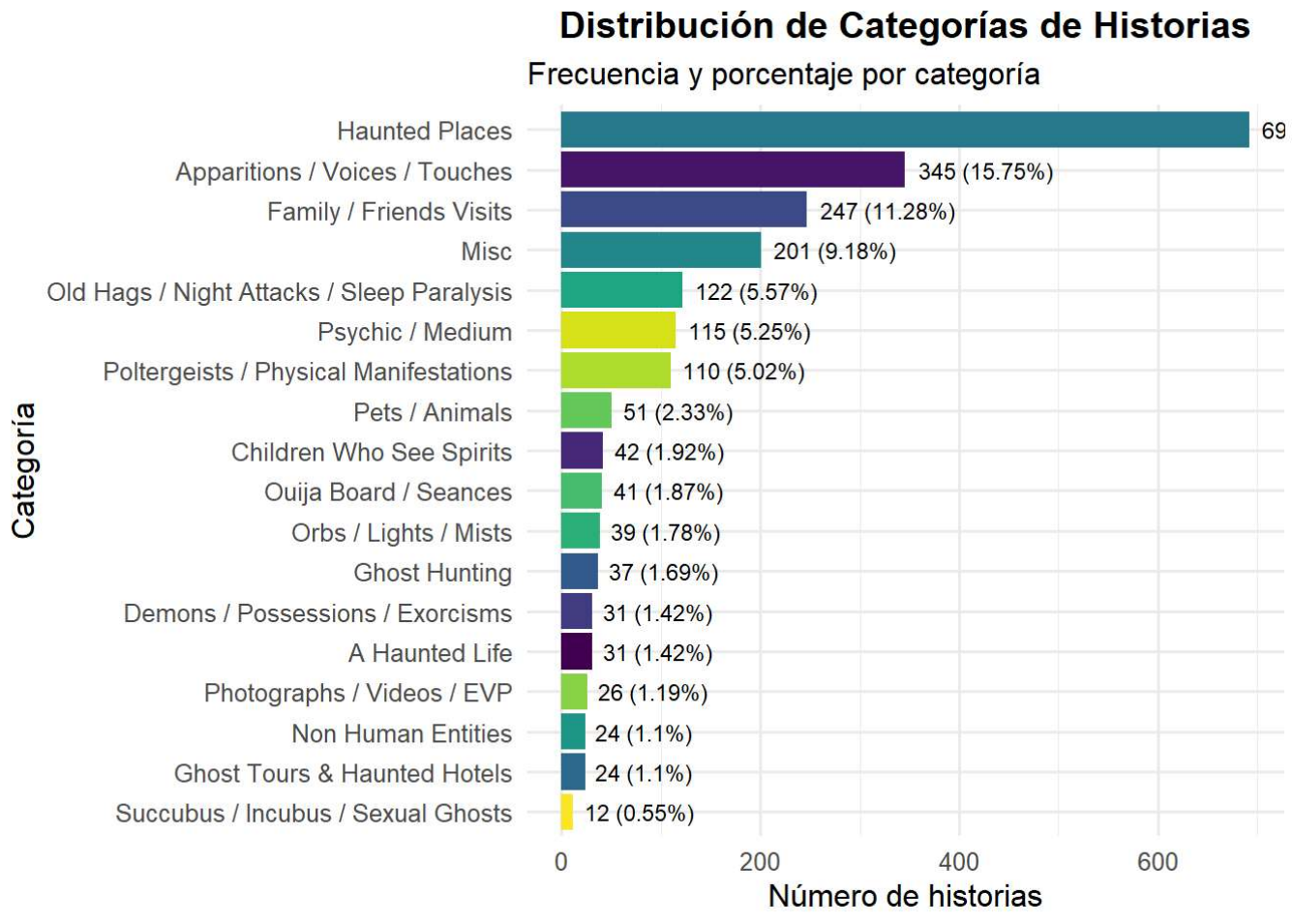
```
=== DISTRIBUCIÓN DE CATEGORÍAS ===
```

```
category_dist <- stories_df %>%
  filter(!is.na(category) & category != "") %>%
  count(category, sort = TRUE) %>%
  mutate(percentage = round(n/sum(n)*100, 2))
print(category_dist)
```

```
                                  category   n percentage
1                            Haunted Places 692      31.60
2             Apparitions / Voices / Touches 345      15.75
3                    Family / Friends Visits 247      11.28
4                                      Misc 201       9.18
5  Old Hags / Night Attacks / Sleep Paralysis 122       5.57
```

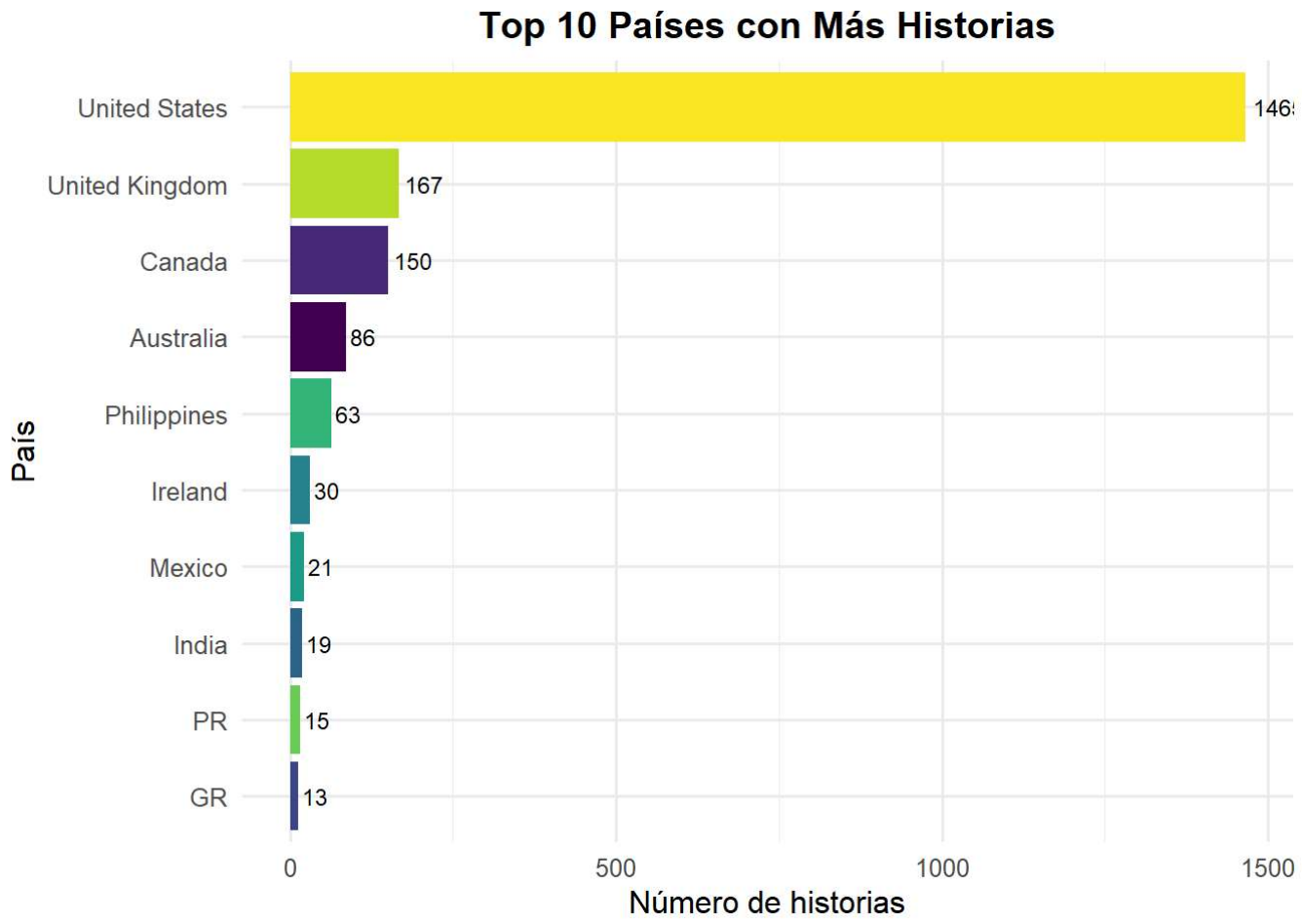
6	Psychic / Medium	115	5.25
7	Poltergeists / Physical Manifestations	110	5.02
8	Pets / Animals	51	2.33
9	Children Who See Spirits	42	1.92
10	Ouija Board / Seances	41	1.87
11	Orbs / Lights / Mists	39	1.78
12	Ghost Hunting	37	1.69
13	A Haunted Life	31	1.42
14	Demons / Possessions / Exorcisms	31	1.42
15	Photographs / Videos / EVP	26	1.19
16	Ghost Tours & Haunted Hotels	24	1.10
17	Non Human Entities	24	1.10
18	Succubus / Incubus / Sexual Ghosts	12	0.55

```r
p1 <- ggplot(category_dist,
             aes(x = reorder(category, n), y = n, fill = category)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = paste0(n, " (", percentage, "%)")), hjust = -0.1, size = 3) +
  coord_flip() +
  scale_fill_viridis_d() +
  labs(title = "Distribución de Categorías de Historias",
       subtitle = "Frecuencia y porcentaje por categoría",
       x = "Categoría", y = "Número de historias") +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
print(p1)
```



```r
if ("location" %in% names(stories_df)) {
  cat("\n=== DISTRIBUCIÓN GEOGRÁFICA ===\n")
  location_dist <- stories_df %>%
    filter(!is.na(location) & location != "") %>%
    count(location, sort = TRUE) %>%
    slice_head(n = 10)
  print(location_dist)

  p2 <- ggplot(location_dist,
               aes(x = reorder(location, n), y = n, fill = location)) +
    geom_col(show.legend = FALSE) +
    geom_text(aes(label = n), hjust = -0.15, size = 3) +
    coord_flip() +
    scale_fill_viridis_d() +
    labs(title = "Top 10 Países con Más Historias",
         x = "País", y = "Número de historias") +
    theme_minimal(base_size = 12) +
    theme(plot.title = element_text(hjust = 0.5, face = "bold"))
  print(p2)
}
```

```
=== DISTRIBUCIÓN GEOGRÁFICA ===
         location    n
1   United States 1465
2  United Kingdom  167
3          Canada  150
4       Australia   86
5     Philippines   63
6         Ireland   30
7          Mexico   21
8           India   19
9              PR   15
10             GR   13
```




```
# ================================================================================
# 3. PROCESAMIENTO DE TEXTO
# ================================================================================

cat("\n=== INICIANDO ANÁLISIS DE TEXTO ===\n")
```

```
=== INICIANDO ANÁLISIS DE TEXTO ===
```

```
# Títulos únicos para servir como ID estable
stories_df <- stories_df %>%
  mutate(title = make.unique(title, sep = "_"))

# Tokenización y limpieza
stories_words <- stories_df %>%
  mutate(doc_id = row_number(),
         story_clean = str_to_lower(story) %>%
                       str_remove_all("[^a-zA-Z\\s]") %>%
                       str_squish()) %>%
  unnest_tokens(word, story_clean) %>%
  filter(nchar(word) > 2) %>%                # descarta tokens muy cortos
  anti_join(stop_words, by = "word")         # quita stopwords

cat("Palabras únicas después de limpieza:",
    length(unique(stories_words$word)), "\n")
```

```
Palabras únicas después de limpieza: 20231
```

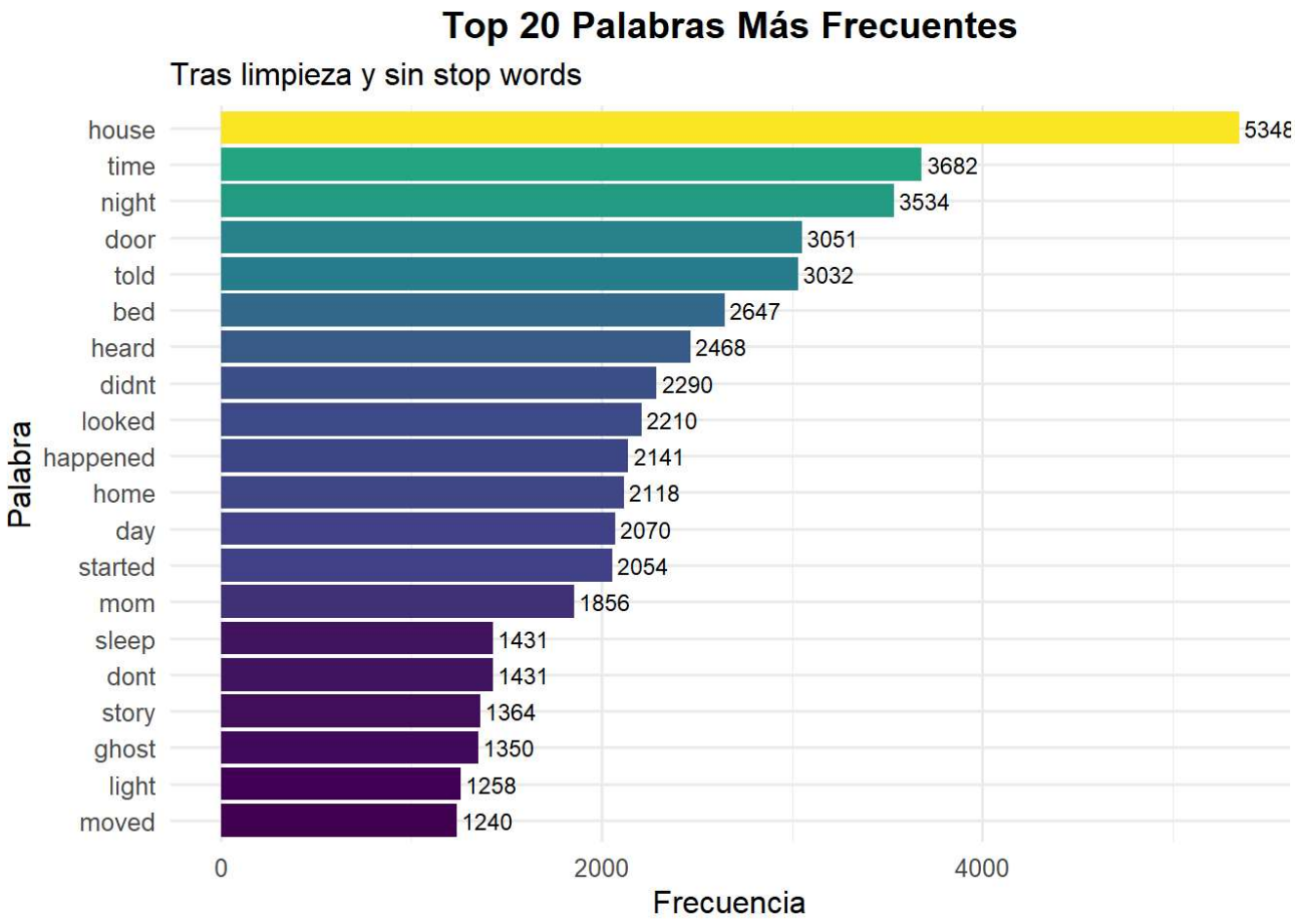
```
# Frecuencias globales
word_freq <- stories_words %>%
  count(word, sort = TRUE)

print("=== TOP 20 PALABRAS MÁS FRECUENTES ===")
```

```
[1] "=== TOP 20 PALABRAS MÁS FRECUENTES ==="
```

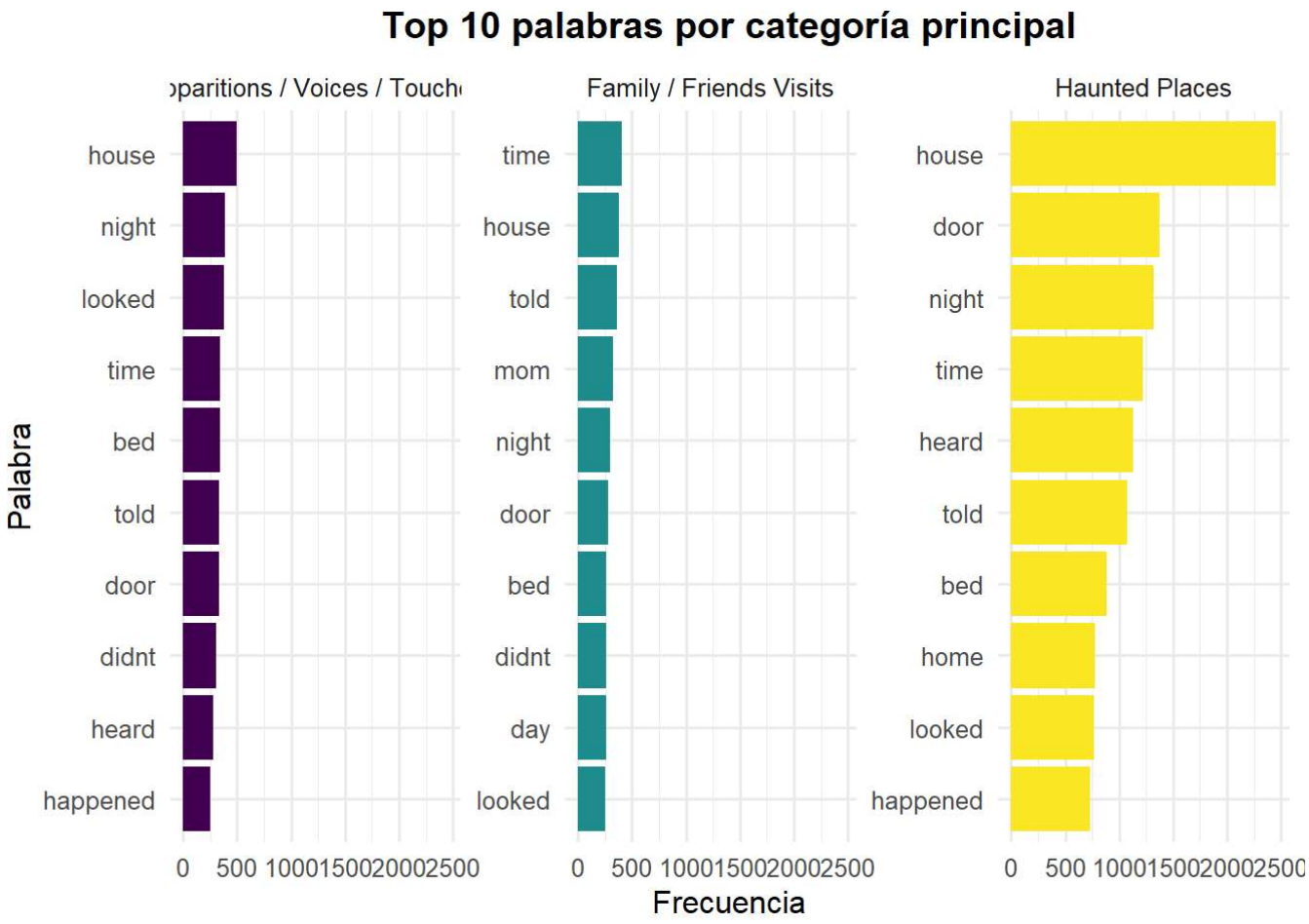
```
print(head(word_freq, 20))
```

```
       word    n
1     house 5348
2      time 3682
3     night 3534
4      door 3051
5      told 3032
6       bed 2647
7     heard 2468
8     didnt 2290
9    looked 2210
10 happened 2141
11     home 2118
12      day 2070
13  started 2054
14      mom 1856
15     dont 1431
16    sleep 1431
17    story 1364
18    ghost 1350
19    light 1258
20    moved 1240
```

```
p3 <- word_freq %>%
  slice_head(n = 20) %>%
  ggplot(aes(x = reorder(word, n), y = n, fill = n)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = n), hjust = -0.1, size = 3) +
  coord_flip() +
  scale_fill_viridis_c() +
  labs(title = "Top 20 Palabras Más Frecuentes",
       subtitle = "Tras limpieza y sin stop words",
       x = "Palabra", y = "Frecuencia") +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
print(p3)
```




```r
# Palabras por categoría (si existe)
if (sum(!is.na(stories_df$category)) > 0) {
  words_by_category <- stories_words %>%
    filter(!is.na(category)) %>%
    count(category, word, sort = TRUE) %>%
    group_by(category) %>%
    slice_head(n = 10) %>%
    ungroup()

  top_categories <- category_dist$category[1:min(3, nrow(category_dist))]

  p4 <- words_by_category %>%
    filter(category %in% top_categories) %>%
    mutate(word = reorder_within(word, n, category)) %>%
    ggplot(aes(y = word, x = n, fill = category)) +
    geom_col(show.legend = FALSE) +
    facet_wrap(~ category, scales = "free_y") +
    scale_y_reordered() +
    scale_fill_viridis_d() +
    labs(title = "Top 10 palabras por categoría principal",
         x = "Frecuencia", y = "Palabra") +
    theme_minimal(base_size = 12) +
    theme(plot.title = element_text(hjust = 0.5, face = "bold"))
  print(p4)
}
```




```
# ================================================================================
# 4. CONSTRUCCIÓN DE MATRIZ SPARSE PROFESIONAL
# ================================================================================

cat("\n=== CONSTRUYENDO MATRIZ DOCUMENT-TERM ===\n")
```

```
=== CONSTRUYENDO MATRIZ DOCUMENT-TERM ===
```

```
# Conteo de palabras por documento
word_counts <- stories_words %>%
  count(title, word, sort = TRUE)

cat("Dimensiones de word_counts:", nrow(word_counts), "filas\n")
```

```
Dimensiones de word_counts: 245331 filas
```

```
# Filtrar palabras muy raras (aparecen en menos de 3 documentos)
word_doc_freq <- word_counts %>%
  group_by(word) %>%
  summarise(doc_freq = n(), .groups = "drop")

common_words <- word_doc_freq %>%
  filter(doc_freq >= 3) %>%
  pull(word)
```

```
cat("Palabras filtradas (≥3 documentos):", length(common_words), "\n")
```

```
Palabras filtradas (≥3 documentos): 7884
```

```
# Crear DTM con palabras filtradas
library(janitor)
dtm_wide <- word_counts %>%
  filter(word %in% common_words) %>%
  rename(doc_id = title) %>%                # <-- CAMBIO: evitar choque con columnas de palabr
  pivot_wider(
    names_from  = word,
    values_from = n,
    values_fill = 0,
    names_repair = "unique",
    names_prefix = "w_"                      # <-- CAMBIO: todas las features de palabras con
  ) %>%
  rename(title = doc_id) %>%                # volvemos a llamar 'title' a la llave del documen
  clean_names()

cat("Dimensiones de DTM:", nrow(dtm_wide), "filas,", ncol(dtm_wide)-1, "características\n")
```

```
Dimensiones de DTM: 2201 filas, 7884 características
```

```
# Agregar información adicional
enhanced_dtm <- dtm_wide %>%
  left_join(stories_df %>% select(title, category, location), by = "title") %>%
  # Agregar características geográficas si están disponibles
  {
    if("location" %in% names(.)) {
      mutate(.,
        has_location = ifelse(is.na(location), 0, 1),
        is_us        = ifelse(location == "United States", 1, 0),
        is_uk        = ifelse(location == "United Kingdom", 1, 0),
        is_canada    = ifelse(location == "Canada", 1, 0)
      ) %>%
      replace_na(list(is_us = 0, is_uk = 0, is_canada = 0, has_location = 0))
    } else {
      .
    }
  }

# Crear matriz sparse
feature_columns <- setdiff(names(enhanced_dtm), c("title", "category", "location"))
dtm_matrix <- enhanced_dtm[, feature_columns, drop = FALSE]

# >>> Solo columnas numéricas a la matriz sparse (evita 'invalid data')
numeric_cols <- vapply(dtm_matrix, is.numeric, logical(1))  # robusto y rápido
dtm_matrix   <- as.matrix(dtm_matrix[, numeric_cols, drop = FALSE])

dtm_sparse <- Matrix(dtm_matrix, sparse = TRUE)
```

```
cat("Matriz sparse creada:", nrow(dtm_sparse), "x", ncol(dtm_sparse), "\n")
```

```
Matriz sparse creada: 2201 x 7888
```

```
cat("Sparsity:", round((1 - nnzero(dtm_sparse)/(nrow(dtm_sparse)*ncol(dtm_sparse)))*100, 2), "
```

```
Sparsity: 98.65 %
```

```
# ========================================================================
# 5. PREPARACIÓN PARA MODELADO (auto-contenido y robusto en Quarto)
# ========================================================================

# Comprobar que la DTM y el data frame base existen
if (!exists("enhanced_dtm") || !exists("dtm_sparse")) {
  stop("Faltan objetos previos: ejecuta los bloques donde se crean `enhanced_dtm` y `dtm_spars
}

# Si no existen `modeling_data` y `target_var`, crearlos aquí
if (!exists("modeling_data") || !exists("target_var")) {
  category_counts <- table(
    enhanced_dtm$category[!is.na(enhanced_dtm$category) & enhanced_dtm$category != ""]
  )
  if (length(category_counts) < 2) {
    stop("No hay suficientes categorías válidas para modelar.")
  }
  max_ratio <- max(category_counts) / min(category_counts)

  if (max_ratio > 5 || min(category_counts) < 20) {
    major_category <- names(which.max(category_counts))
    modeling_data <- enhanced_dtm %>%
      dplyr::filter(!is.na(category) & category != "") %>%
      dplyr::mutate(category_binary = dplyr::if_else(category == major_category, major_categor
      dplyr::mutate(category_binary = factor(category_binary))
    target_var <- "category_binary"
  } else {
    modeling_data <- enhanced_dtm %>%
      dplyr::filter(!is.na(category) & category != "") %>%
      dplyr::mutate(category = factor(category))
    target_var <- "category"
  }
}

# Alinear filas de la DTM con los títulos de `modeling_data`
row_idx  <- match(modeling_data$title, enhanced_dtm$title)
present  <- !is.na(row_idx)
if (!all(present)) {
  message("Advertencia: se descartan ", sum(!present),
          " documentos de modeling_data que no están en la DTM.")
  modeling_data <- modeling_data[present, , drop = FALSE]
  row_idx <- row_idx[present]
}
```

```r
# Extraer X (matriz de características) y y (objetivo)
X_all_sparse <- dtm_sparse[row_idx, , drop = FALSE]
y_all        <- modeling_data[[target_var]]

# Convertir a matriz base (evita errores de subíndices S4 en algunos métodos)
X_all <- as.matrix(X_all_sparse)

# Split estratificado 70/30
set.seed(1234)
train_idx <- caret::createDataPartition(y_all, p = 0.7, list = FALSE)

X_train <- X_all[train_idx, , drop = FALSE]
X_test  <- X_all[-train_idx, , drop = FALSE]
y_train <- y_all[train_idx]
y_test  <- y_all[-train_idx]

# Resumen
cat("Tamaño X_train:", nrow(X_train), "x", ncol(X_train), "\n")
```

```
Tamaño X_train: 1534 x 7888
```

```r
cat("Tamaño X_test :", nrow(X_test),  "x", ncol(X_test),  "\n")
```

```
Tamaño X_test : 656 x 7888
```

```r
cat("Clases en train:\n"); print(table(y_train))
```

```
Clases en train:

y_train
Haunted Places          Other
           485           1049
```

```r
cat("Clases en test:\n");  print(table(y_test))
```

```
Clases en test:

y_test
Haunted Places          Other
           207            449
```

```r
# ============================================================================
# 6. MODELOS BASE RÁPIDOS (sparse)
# ============================================================================

cat("\n=== ENTRENANDO MODELOS NAIVE BAYES (rápidos, matriz sparse) ===\n")
```

```
=== ENTRENANDO MODELOS NAIVE BAYES (rápidos, matriz sparse) ===
```

```r
# Modelo NB Gaussiano con Laplace=1 (base)
nb_norm <- naive_bayes(x = X_train, y = y_train, laplace = 1)

# Modelo NB Poisson con Laplace=1 (recomendado para conteos)
nb_pois <- naive_bayes(x = X_train, y = y_train, laplace = 1, usepoisson = TRUE)

# Predicciones y métricas
pred_norm <- predict(nb_norm, X_test)
pred_pois <- predict(nb_pois, X_test)

extract_metrics <- function(pred, actual, name){
  cm <- confusionMatrix(pred, actual)
  if (length(levels(actual)) == 2) {
    prec <- cm$byClass["Pos Pred Value"]; rec <- cm$byClass["Sensitivity"]; f1 <- cm$byClass["
  } else {
    prec <- mean(cm$byClass[,"Pos Pred Value"], na.rm = TRUE)
    rec  <- mean(cm$byClass[,"Sensitivity"], na.rm = TRUE)
    f1   <- mean(cm$byClass[,"F1"], na.rm = TRUE)
  }
  data.frame(Model = name,
             Accuracy = round(cm$overall["Accuracy"], 4),
             Precision = round(prec, 4),
             Recall = round(rec, 4),
             F1_Score = round(f1, 4))
}

metrics_norm <- extract_metrics(pred_norm, y_test, "NB (Gauss) + Laplace=1")
metrics_pois <- extract_metrics(pred_pois, y_test, "NB (Poisson) + Laplace=1")
initial_metrics <- bind_rows(metrics_norm, metrics_pois)

print("=== MÉTRICAS INICIALES (modelos rápidos) ===")
```

```
[1] "=== MÉTRICAS INICIALES (modelos rápidos) ==="
```

```r
print(knitr::kable(initial_metrics, caption = "Comparación de modelos base"))
```

Table: Comparación de modelos base

	Model	Accuracy	Precision	Recall	F1_Score
Accuracy...1	NB (Gauss) + Laplace=1	0.3155	0.3155	1	0.4797
Accuracy...2	NB (Poisson) + Laplace=1	0.3155	0.3155	1	0.4797

```r
# ============================================================================
# 7. VALIDACIÓN CRUZADA LIGERA PARA NB-POISSON
# ============================================================================

cat("\n=== OPTIMIZACIÓN LIGERA CON CROSS-VALIDATION (NB-Poisson) ===\n")
```

```
=== OPTIMIZACIÓN LIGERA CON CROSS-VALIDATION (NB-Poisson) ===
```

```
laplace_grid <- c(1)   # grid reducido
cv_folds     <- 3      # 3-fold CV para rapidez

folds <- createFolds(y_train, k = cv_folds)

perform_cv_sparse <- function(laplace_val){
  acc <- numeric(cv_folds)
  for (i in seq_along(folds)) {
    tr <- setdiff(seq_len(nrow(X_train)), folds[[i]])
    va <- folds[[i]]
    mdl <- naive_bayes(x = X_train[tr, ], y = y_train[tr],
                       laplace = laplace_val, usepoisson = TRUE)
    prd <- predict(mdl, X_train[va, ])
    acc[i] <- mean(prd == y_train[va])
  }
  data.frame(Laplace = laplace_val,
             Mean_Accuracy = mean(acc),
             SD_Accuracy   = sd(acc),
             Poisson = TRUE)
}

cv_results <- purrr::map_df(laplace_grid, perform_cv_sparse) %>%
  arrange(desc(Mean_Accuracy)) %>%
  mutate(Rank = row_number())

print("=== RESULTADOS DE CROSS-VALIDATION (NB-Poisson) ===")
```

```
[1] "=== RESULTADOS DE CROSS-VALIDATION (NB-Poisson) ==="
```

```
print(knitr::kable(cv_results, caption = "Resultados 3-fold CV (grid reducido)"))
```

```
Table: Resultados 3-fold CV (grid reducido)

| Laplace| Mean_Accuracy| SD_Accuracy|Poisson | Rank|
|-------:|-------------:|-----------:|:-------|----:|
|       1|     0.3161663|   0.0004157|TRUE    |    1|
```

```
# ============================================================================
# 8. MODELO FINAL
# ============================================================================

cat("\n=== ENTRENANDO MODELO FINAL OPTIMIZADO ===\n")
```

```
=== ENTRENANDO MODELO FINAL OPTIMIZADO ===
```

```
best_params <- cv_results[1, ]
cat("Mejores parámetros:\n")
```

```
Mejores parámetros:
```

```
cat("  Laplace:", best_params$Laplace, "\n")
```

```
  Laplace: 1
```

```
cat("  Usar Poisson: TRUE\n")
```

```
  Usar Poisson: TRUE
```

```
cat("  CV Accuracy:", round(best_params$Mean_Accuracy, 4), "±",
    round(best_params$SD_Accuracy, 4), "\n")
```

```
  CV Accuracy: 0.3162 ± 4e-04
```

```
nb_final <- naive_bayes(
  x = X_train, y = y_train,
  laplace = best_params$Laplace, usepoisson = TRUE
)

pred_final <- predict(nb_final, X_test)
cm_final   <- confusionMatrix(pred_final, y_test)

metrics_final <- extract_metrics(pred_final, y_test,
                                 paste0("NB-Poisson (Laplace=", best_params$Laplace, ")"))

print("=== MODELO FINAL ===")
```

```
[1] "=== MODELO FINAL ==="
```

```
print(knitr::kable(metrics_final, caption = "Métricas del modelo final"))
```

Table: Métricas del modelo final

	Model	Accuracy	Precision	Recall	F1_Score
Accuracy	NB-Poisson (Laplace=1)	0.3155	0.3155	1	0.4797

```
# ============================================================================
# 9. ANÁLISIS DE RESULTADOS
# ============================================================================

cat("\n=== ANÁLISIS DETALLADO DE RESULTADOS ===\n")
```

```
=== ANÁLISIS DETALLADO DE RESULTADOS ===
```

```
print("Matriz de Confusión del Modelo Final:")
```

```
[1] "Matriz de Confusión del Modelo Final:"
```

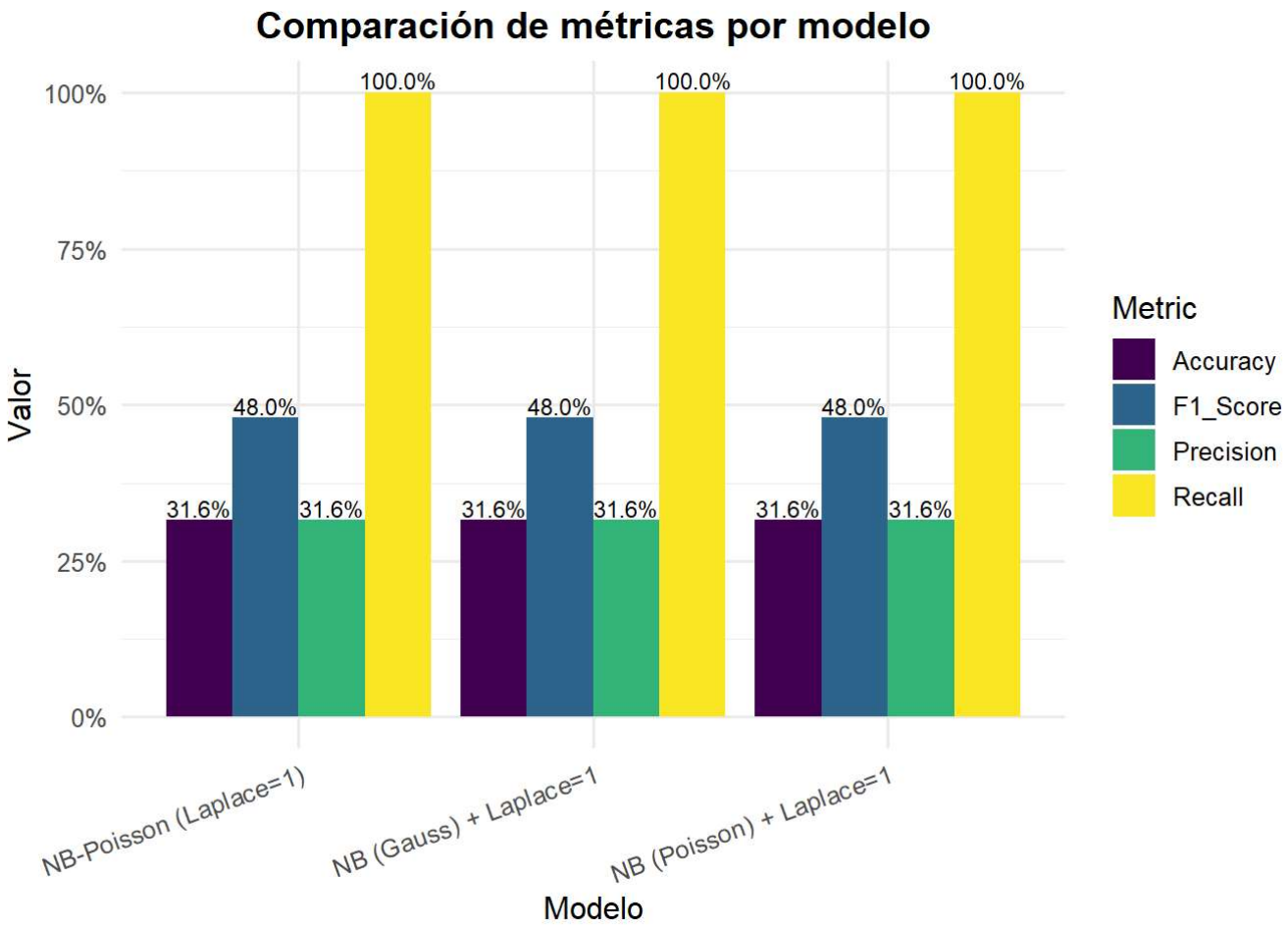
```
print(cm_final$table)
```

```
                Reference
Prediction       Haunted Places Other
  Haunted Places            207   449
  Other                       0     0
```

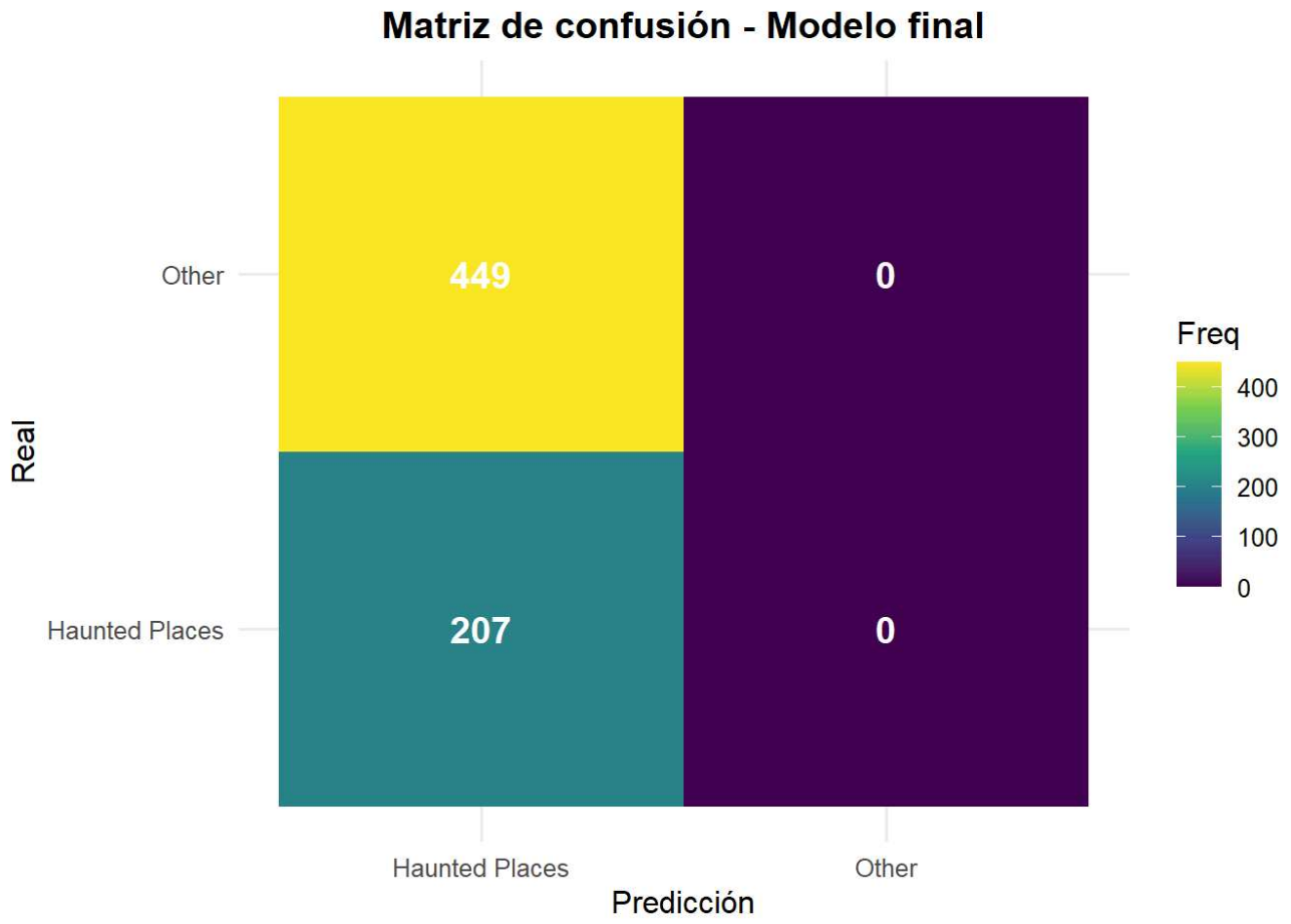
```
if (length(levels(y_test)) > 2) {
  class_metrics <- data.frame(
    Class       = rownames(cm_final$byClass),
    Sensitivity = round(cm_final$byClass[,"Sensitivity"], 4),
    Specificity = round(cm_final$byClass[,"Specificity"], 4),
    Precision   = round(cm_final$byClass[,"Pos Pred Value"], 4),
    F1          = round(cm_final$byClass[,"F1"], 4)
  )
  print(knitr::kable(class_metrics, caption = "Métricas por clase"))
} else {
  print("Clasificación binaria: métricas globales mostradas arriba.")
}
```

```
[1] "Clasificación binaria: métricas globales mostradas arriba."
```

```
# Comparación visual de métricas
comparison_plot <- bind_rows(initial_metrics, metrics_final) %>%
  select(Model, Accuracy, Precision, Recall, F1_Score) %>%
  pivot_longer(cols = -Model, names_to = "Metric", values_to = "Value") %>%
  ggplot(aes(x = Model, y = Value, fill = Metric)) +
  geom_col(position = "dodge") +
  geom_text(aes(label = scales::percent(Value, accuracy = 0.1)),
            position = position_dodge(width = 0.9), vjust = -0.2, size = 3) +
  scale_fill_viridis_d() +
  scale_y_continuous(labels = percent_format()) +
  labs(title = "Comparación de métricas por modelo",
       x = "Modelo", y = "Valor") +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x  = element_text(angle = 20, hjust = 1))
print(comparison_plot)
```




```r
# Matriz de confusión gráfica (si #clases es pequeño)
if (length(levels(y_test)) <= 4) {
  cm_plot_data <- as.data.frame(cm_final$table) %>%
    rename(Actual = Reference, Predicted = Prediction)

  cm_plot <- ggplot(cm_plot_data, aes(x = Predicted, y = Actual, fill = Freq)) +
    geom_tile() +
    geom_text(aes(label = Freq), color = "white", size = 5, fontface = "bold") +
    scale_fill_viridis_c() +
    labs(title = "Matriz de confusión - Modelo final",
         x = "Predicción", y = "Real") +
    theme_minimal(base_size = 12) +
    theme(plot.title = element_text(hjust = 0.5, face = "bold"))
  print(cm_plot)
}
```




```
# ============================================================================
# 10. REPORTE EJECUTIVO (consola)
# ============================================================================

cat("\n", paste(rep("=", 80), collapse = ""), "\n")
```

```
 ================================================================================
```

```
cat("REPORTE EJECUTIVO FINAL\n")
```

```
REPORTE EJECUTIVO FINAL
```

```
cat(paste(rep("=", 80), collapse = ""), "\n")
```

```
================================================================================
```

```
cat("\nRESUMEN DEL DATASET:\n")
```

```
RESUMEN DEL DATASET:
```

```
cat("  • Total de historias analizadas:", nrow(stories_df), "\n")
```

- Total de historias analizadas: 2201

```
cat("  • Historias con categoría válida:", nrow(modeling_data), "\n")
```

- Historias con categoría válida: 2190

```
if ("location" %in% names(stories_df)) {
  cat("  • Historias con ubicación:", sum(!is.na(stories_df$location)), "\n")
}
```

- Historias con ubicación: 2201

```
cat("  • Características textuales (DTM):", ncol(dtm_sparse), "\n")
```

- Características textuales (DTM): 7888

```
cat("  • Tipo de clasificación:", ifelse(target_var == "category_binary", "Binaria", "Multicla
```

- Tipo de clasificación: Binaria

```
cat("\nPROCESAMIENTO DE TEXTO:\n")
```

PROCESAMIENTO DE TEXTO:

```
cat("  • Palabras únicas tras limpieza:", length(unique(stories_words$word)), "\n")
```

- Palabras únicas tras limpieza: 20231

```
cat("  • Palabras usadas (≥3 docs):", length(common_words), "\n")
```

- Palabras usadas (≥3 docs): 7884

```
cat("  • Sparsity de la matriz:", round((1 - nnzero(dtm_sparse)/(nrow(dtm_sparse)*ncol(dtm_spa
```

- Sparsity de la matriz: 98.65 %

```
cat("\nMODELADO:\n")
```

MODELADO:

```
cat("  • Algoritmo: Naive Bayes (Poisson) con Laplace=", best_params$Laplace, "\n", sep = "")
```

- Algoritmo: Naive Bayes (Poisson) con Laplace=1

```
cat("  • Validación: 3-fold CV (grid reducido)\n")
```

- Validación: 3-fold CV (grid reducido)

```
cat("  • División train/test: 70% / 30%\n")
```

- División train/test: 70% / 30%

```
cat("\nRESULTADOS FINALES (test):\n")
```

RESULTADOS FINALES (test):

```
cat("  • Accuracy:", sprintf("%.2f%%", metrics_final$Accuracy * 100), "\n")
```

- Accuracy: 31.55%

```
cat("  • Precision:", sprintf("%.2f%%", metrics_final$Precision * 100), "\n")
```

- Precision: 31.55%

```
cat("  • Recall:", sprintf("%.2f%%", metrics_final$Recall * 100), "\n")
```

- Recall: 100.00%

```
cat("  • F1-Score:", sprintf("%.2f%%", metrics_final$F1_Score * 100), "\n")
```

- F1-Score: 47.97%

```
cat("\nCONCLUSIÓN:\n")
```

CONCLUSIÓN:

```
if (metrics_final$Accuracy > 0.8) {
  cat("  • Desempeño excelente para el objetivo planteado.\n")
} else if (metrics_final$Accuracy > 0.7) {
  cat("  • Buen desempeño, con margen de mejora mediante ajuste de features.\n")
} else {
  cat("  • Desempeño moderado; conviene refinar vocabulario y balance de clases.\n")
}
```

- Desempeño moderado; conviene refinar vocabulario y balance de clases.

```
cat("\nInformación de la sesión:\n")
```

Información de la sesión:

```
print(sessionInfo())
```

```
R version 4.5.1 (2025-06-13 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26100)

Matrix products: default
  LAPACK version 3.12.1

locale:
[1] LC_COLLATE=Spanish_Mexico.utf8  LC_CTYPE=Spanish_Mexico.utf8
LC_MONETARY=Spanish_Mexico.utf8
[4] LC_NUMERIC=C                    LC_TIME=Spanish_Mexico.utf8

time zone: America/Mexico_City
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] scales_1.4.0      viridis_0.6.5     viridisLite_0.4.2 janitor_2.2.1     e1071_1.7-16
 [6] naivebayes_1.0.0  caret_7.0-1       lattice_0.22-7    Matrix_1.7-3      tidytext_0.4.3
[11] lubridate_1.9.4   forcats_1.0.0     stringr_1.5.2     dplyr_1.1.4       purrr_1.1.0
[16] readr_2.1.5       tidyr_1.3.1       tibble_3.3.0      ggplot2_3.5.2     tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] tidyselect_1.2.1     timeDate_4041.110    farver_2.1.2         fastmap_1.2.0
janeaustenr_1.0.0
 [6] pROC_1.19.0.1        digest_0.6.37        rpart_4.1.24         timechange_0.3.0
lifecycle_1.0.4
[11] tokenizers_0.3.0     survival_3.8-3       magrittr_2.0.3       compiler_4.5.1
rlang_1.1.6
[16] tools_4.5.1          yaml_2.3.10          data.table_1.17.8    knitr_1.50
labeling_0.4.3
[21] plyr_1.8.9           RColorBrewer_1.1-3   withr_3.0.2          nnet_7.3-20
grid_4.5.1
[26] stats4_4.5.1         future_1.67.0        globals_0.18.0       iterators_1.0.14
MASS_7.3-65
[31] cli_3.6.5            rmarkdown_2.29       generics_0.1.4       rstudioapi_0.17.1
future.apply_1.20.0
[36] reshape2_1.4.4       tzdb_0.5.0           proxy_0.4-27         splines_4.5.1
parallel_4.5.1
[41] vctrs_0.6.5          hardhat_1.4.2        jsonlite_2.0.0       hms_1.1.3
listenv_0.9.1
[46] foreach_1.5.2        gower_1.0.2          recipes_1.3.1        glue_1.8.0
parallelly_1.45.1
[51] codetools_0.2-20     stringi_1.8.7        gtable_0.3.6         pillar_1.11.0
htmltools_0.5.8.1
[56] ipred_0.9-15         lava_1.8.1           R6_2.6.1             evaluate_1.0.5
SnowballC_0.7.1
[61] snakecase_0.11.1     class_7.3-23         Rcpp_1.1.0           gridExtra_2.3
```

```
    nlme_3.1-168
    [66] prodlim_2025.04.28   xfun_0.53           pkgconfig_2.0.3     ModelMetrics_1.2.2.2
```